

Quantitative Analysis of Services

Igor Cappello¹, Allan Clark², Stephen Gilmore², Diego Latella³, Michele Loreti⁴,
Paola Quaglia¹, and Stefano Schivo¹

Dipartimento di Ingegneria e Scienza dell'Informazione, Università di Trento¹

School of Informatics, The University of Edinburgh²

Istituto di Scienza e Tecnologie dell'Informazione "A. Faedo"- CNR³

Dipartimento di Sistemi e Informatica, Università di Firenze⁴

Abstract. We show a number of applications of the tools which have been developed within the *SENSORIA* project to perform quantitative analysis of services. These tools are formally grounded on source calculi which allow the description of services at distinct levels of abstraction, and hence pose distinct challenges to both modelling and analysis.

The reported applications refer to (suitable subcomponents of) the Finance Case-Study, and show instances of, respectively, exact model checking of *MarCaSPiS* against the both state-aware and action-aware logic *SoSL*, exact and statistical model checking of *sCOWS* against the state-aware logic *CSL*, querying of *PEPA* models by terms of the *XSP* language that expresses both state-aware and action-aware stochastic probes.

1 Introduction

A number of calculi have been defined to formally model services and then analyze their behaviour. Some of them come equipped with quantitative measures and probabilistic/stochastic semantics, and hence allow the description of non-functional aspects of computation like, e.g., performance, resource usage, or dependability.

Based on the above calculi, some tools have been developed within the *SENSORIA* Project to assist in the verification of the quantitative properties of services. Here we show the applicability of those tools together with a sample of the obtained results. The analysis refers to a credit request scenario. This in turn is part of the specification of a credit portal which is referred to as the *SENSORIA* Finance Case-Study and is fully illustrated in Chapter 7-4.

The credit portal is a business-to-business (B2B) application where a bank wishes to offer its loan services to business customers. It is important that loan applications are processed efficiently (so that the bank does not lose customers to a rival bank). A high degree of automation is provided to meet this demand for efficient processing. Where human intervention is needed in the decision-making process this too must be driven by a deep sense of urgency in processing the request in a timely manner.

The credit portal is implemented using the *JBOSS* enterprise Java technology. This is an extensible Web service framework in which traditional web service components can be supplemented using flexible rulesets which allow complex business rules to be coded concisely. These rulesets can also be updated quickly in response to changing

business policies, regulations, procedures, and governmental or legal requirements. This configuration allows the bank both to respond quickly to changing practices within the financial industry and to gain the greatest advantage from business opportunities by bringing new products and services onto the market ahead of its rivals.

Some businesses which use the services of the bank are long-established customers with a lengthy and well-documented history of financial solidity and probity, together with substantial credit reserves and securities. Modest loan applications from these customers can be directly approved by a *pre-decision* process which validates the content of the loan request and the securities. In such a case it is possible that the decision to lend can be taken entirely by the application of the predefined rules stored on the server and executed by the local rule engine. This is the fastest route to approval of a loan request because it is one which completes entirely without human intervention.

Not all loan requests can be so rapidly approved. Many will require more lengthy scrutiny, evaluation and checking. In these cases a bank clerk will process the credit request. There are several possible next outcomes here. First, the clerk may approve the request but must then forward the credit request to a supervisor who must also approve the request. Second, the clerk may decline the request. Third, the clerk may enter into a negotiation with the customer with the intention of updating the request to reduce the capital requested, or change the terms of repayment. This then initiates another request from the customer which is to be processed in the manner just described.

Quantitative evaluation of this scenario presents technical challenges because there are two distinct timescales in use here. The pre-decision processing is performed on a timescale of seconds whereas the human processing and evaluation proceeds on a timescale of days. Systems where activities proceed at very different rates provide numerical problems for the underlying software routines because they demand more computational effort than models which operate on a single timescale. When rates are separated by several orders of magnitude, as they are here, the problem is often referred to as a *multi-scale problem* and its attendant numerical problem as *stiff* (meaning intractable or computationally expensive). Thus this problem is a good test for the robustness of the quantitative analysis tools developed in the SENSORIA Project.

The rest of the chapter is organized as follows. Section 2, Section 3, and Section 4 present the analysis of the credit request scenario specified in MarCaSPiS, in sCOWS [11, 13], and in PEPA [6], respectively. Concluding remarks are reported in Section 5.

The analysis of the MarCaSPiS specification of the scenario is based on a stochastic model checking technique. Properties are described in SoSL (Service-Oriented Stochastic Logic), a stochastic modal logic for service-oriented computing presented in Chapter 5-1. The analysis is then performed by exploiting SoSL-MC, a stochastic model-checker for SoSL which uses MRMC [7] as a component. In particular MRMC is wrapped in the algorithms implemented in SoSL-MC.

Section 3 presents the sCOWS specification of the credit request scenario. The model is analyzed by means of two different techniques: exact model checking and statistical simulation-based model checking. In both cases, the property specification language is CSL [1]. The tool for statistical model checking is a stand-alone application, while the one for exact model checking provides input to PRISM [8], a well-known

probabilistic model checker. Comparisons between the results obtained by the two kinds of analysis are also reported.

The applications shown in Section 4 are based on the PEPA specification of the relevant scenario. Stochastic properties of services are obtained by querying the model using the XSP performance language [4]. This is an action-aware language of stochastic probes: once fixed the actions relevant to the sort of analysis that the user wishes to carry on, the tool delivers the probability of completing the passage between them in the given time bound.

2 Analysis in MarCaSPiS

MarCaSPiS is the Markovian extension of CaSPiS [2]. In MarCaSPiS, each *output activity* (service invocation, concretion and return) is equipped with a parameter (a *rate*, $\lambda \in \mathbb{R}^+$) characterising a random variable with a negative exponential distribution, modeling the duration of the activity. Furthermore, each *input activity* (service definition and abstractions) is annotated with a *weight* ($\omega \in \mathbb{N}^+$): a positive integer that will be used for determining the probability that the specific input is selected when a complementary output is executed.

In this section we show how MarCaSPiS can be used for specifying and verifying quantitative aspects of the Finance Case-Study. A MarCaSPiS specification of the Finance Case-Study is obtained from the one in CaSPiS, presented in Chapter 2-1, by adding rates (and weights) to actions.

Parties considered in the specification are:

- the **supervisor** and **clerk** that can be asked to review the request;
- the **portal** that acts as an interface for clients and that coordinates the whole procedure;
- auxiliary services, **login** and **rating**, invoked by the portal in order to assist in the request evaluation.

Each of the above parties is rendered as a service that is waiting for an invocation. We let **portal**, **login** and **rating** be replicated services. Hence, they are always able to react to invocations. On the contrary, **supervisor** and **clerk**, which describe the behaviour of people involved in the decision process, are not replicated.

A client that wants to submit a credit request has to first log into the system. If the login is successful then he will be asked for some credit data (e.g. the amount) and for some balance and security guarantees. Then the request review phase starts. A rating is computed for the client: either the credit is immediately granted (rating AAA) or the intervention of either a clerk or a supervisor is required (rating BBB and CCC). The decision (decline or accept) is then communicated to the client.

In the analysis proposed in this section we study how system performance is influenced by the client distributions within the three rating categories. The following scenarios are considered:

S1: 50% of clients have rate AAA, 33% rate BBB, and 17% rate CCC.

S2: 25% of clients have rate AAA, 50% rate BBB, and 25% rate CCC.

S3: 17% of clients have rate AAA, 33% rate BBB, and 50% rate CCC.

In the analysis the following properties will be considered:

1. *System performance*, how fast the system is able to handle pending requests;
2. *Supervisor and Clerk workload*, how much time supervisor and clerk spend to review requests;
3. *System reactivity*, how much time a system needs to manage a request.

These properties are specified by means of SoSL (Service-Oriented Stochastic Logic). This is a *temporal logic* that permits describing the dynamic evolution of the system. SoSL is both *action-* and *state-*based and it is equipped with primitives that permits the use of real-time bounds in the logical characterisation of the behaviours of interest. Moreover, SoSL is a *probabilistic logic* that permits expressing not only functional properties, but also properties related to performance and dependability aspects.

In the analysis we assume that, on the average, `portal` takes a decision every time unit, `clerk` takes a decision every 5 time units, and `supervisor` takes a decision every 10 time units. Moreover, we also assume that `login` and `rating` handle 10 invocations per time units on the average.

2.1 System performance

We aim at verifying the capability of the system to handle pending requests. For this reason, we consider a configuration where the credit portal is in parallel with n clients ($n \in \{1, 2, 3\}$). We are interested in determining the probability (p) that all the clients are served within t time units (All the clients are served when the system reaches a deadlock state). This property can be expressed in SoSL by the following formula:

$$\mathcal{P}_{\geq p}(\text{TRUE} \text{ } \tau \text{ } \mathcal{U}^{< t} [\top] \text{FALSE})$$

In Figure 2.1 is shown how the probability p varies as a function of time t . In all the considered cases (1, 2 and 3 clients) the greater is the percentage of clients with rating AAA, the higher is the probability of serving all the requests within t time units.

2.2 Supervisor and Clerk workload

Besides system performance, it could be also interesting to verify the workload of supervisor and clerk. This analysis could help system designers to evaluate the number of people to engage.

In order to perform this analysis we consider a system where a client continuously contacts the portal. In this case we are interested in studying the probability that services supervisor and clerk are under-used. We say that a service is *under-used* if the probability is less than 0.4 that the service is used within 10 time units. Under-utilization for service s can be rendered in SoSL with formula Φ_s defined as follows:

$$\Phi_s \triangleq \mathcal{P}_{< 0.4}(\text{TRUE} \text{ } \tau \text{ } \mathcal{U}_{\leftarrow s}^{< 10} \text{TRUE})$$

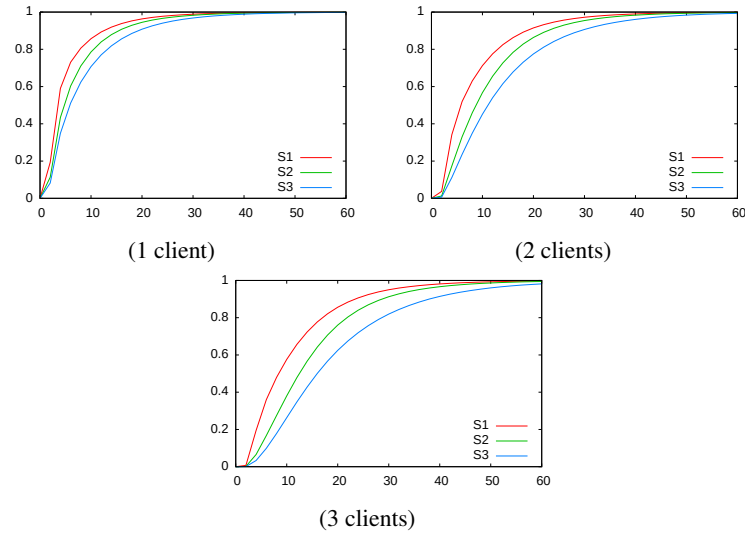


Fig. 1. System performance

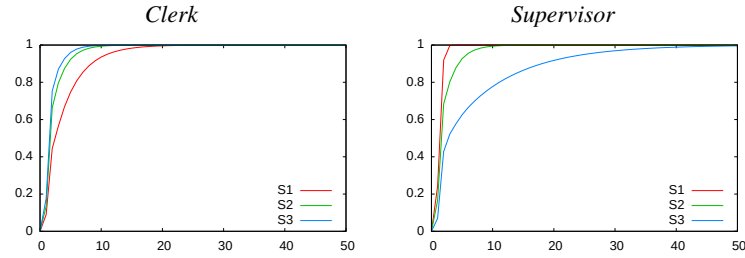


Fig. 2. Supervisor and Clerk workload

We are interested in those states such that the probability is larger than or equal to p that, by time t , a state is reached where supervisor (resp. clerk) is *under-used*.

These properties are rendered in SoSL as follows:

$$\mathcal{P}_{\geq p}(\text{TRUE} \text{ } \top \text{ } \mathcal{U}^{<t} \Phi_{\text{supervisor}}) \quad \mathcal{P}_{\geq p}(\text{TRUE} \text{ } \top \text{ } \mathcal{U}^{<t} \Phi_{\text{clerk}})$$

In Figure 2.2 is reported the result of the analysis. In the considered specification both the clerk and the supervisor are under-used.

2.3 System reactivity

In Section 2.2 we have studied a performance measure of the Finance Case-Study. This is expressed in terms of the probability to completely handle a set of clients in a given

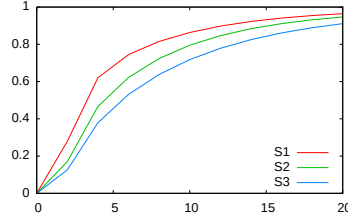


Fig. 3. System reactivity

time with a given probability. However, this measure is also influenced by many aspects that are not under the control of the designer. For instance, the time spent by a client for sending data.

In this section we study *system reactivity*. This aims at measuring the capacity of the system to react to external interactions. In particular, we will consider a property of the form: “if a client contacts the portal and sends the data, then with probability p the portal will send the decision within t time units”.

This property can be specified in SoSL as follows:

$$\nabla x. \{s(x) \cdot x : \langle \text{ID} \rangle \cdot x : \langle \text{AMNT} \rangle\}_{\geq 1} \mathcal{P}_{\geq p}(\text{TRUE} \rightarrow \mathcal{U}^{<t} \{s(x) \cdot x : \langle \text{DEC} \rangle\}_{\geq 1} \text{TRUE})$$

In Figure 2.3 is reported the result of the analysis where, as expected, we have better performance in scenario **S1** while the worst case is scenario **S3**.

3 Analysis in sCOWS

In this section we report on some analysis results obtained by using two tools which have been developed within the SENSORIA Project to verify quantitative properties of services described in a stochastic extension of COWS [9]. The detailed description of the stochastic calculus, named sCOWS, is outside the scope of this paper. Here we just mention that, in the tradition of stochastic process calculi, the main syntactic difference between sCOWS and COWS is in the fact that in sCOWS basic actions are associated with a random variable expressing their rates. Other differences - i.e., the use of service identifiers vs replication, and the adoption of a labelled semantics vs a reduction semantics - are ascribed to meet some technical requirements for the applicability of Markovian techniques. For a detailed description of the sCOWS semantics the interested reader is referred to either [11] (which shows the essence of the stochastic enrichment on the monadic calculus) or [13] (where the extension of the full polyadic calculus is presented).

COWS is a calculus strongly inspired by WS-BPEL which combines primitives of well-known process calculi (like, e.g., the π -calculus [10, 12]) with constructs for web services orchestration. Differently from CASPiS, no primitive for opening or ending sessions is explicitly available in COWS. The communication paradigm of the calculus

is rather based on a mechanism of *best-matching* of the parameters of complementary invoke (send) and request (receive) activities. Briefly put, when an invoke (request, resp.) can match more than one request (invoke, resp.), only the communications that induce the least number of parameter actualizations are allowed. For instance, using an asynchronous π -calculus like notation and assuming that integer numbers play the role of ground constants, in the following parallel composition

$$\bar{x}\langle x, 7 \rangle \mid x(w, 7).P_1 \mid x(y, z).P_2 \mid x(z, 11).P_3$$

one single communication would be allowed: the one between the sending process $\bar{x}\langle x, 7 \rangle$ and the receiving process $x(w, 7).P_1$. Precisely, the pairing of $\bar{x}\langle x, 7 \rangle$ with $x(z, 11)$ would be made impossible by the mismatch of the ground constants, and the pairing of $\bar{x}\langle x, 7 \rangle$ with $x(y, z)$ would instead be prevented by the fact that this is not the best available match for the output action. Indeed, pairing $\bar{x}\langle x, 7 \rangle$ with $x(y, z)$ would induce the substitution of both y and z , while pairing $\bar{x}\langle x, 7 \rangle$ with $x(w, 7)$ results in substituting w only. The above example simply shows how the mechanism of sessioning is rendered in COWS. Just think of the ground constant 7 as a private session identifier: all the communications reserved for that particular session will not be intercepted by concurrent peers.

3.1 sCOWS specification of the scenario

We present below the sCOWS specification of the analyzed scenario. The setting is of a bank credit request service, which can be invoked by customers. The example presents the login phase of the scenario, which is implemented at two distinct levels of abstraction. In the first case, referred to as *fine-grained*, the bank service is composed of a portal service (acting as an interface towards customers) and a login database (which may be thought of as a bank private service). In the second case, called *coarse-grained*, the login database is not explicitly considered: the whole login process is modelled as if it were completely handled by the portal implementation, and the existence of a login database is abstracted away. Indeed, the two specifications interpret the login process as seen by the bank and by the customer, respectively. The services involved in the two cases are, respectively:

- Customer (Table 1) and Portal (Table 2) for the coarse-grained instance;
- Customer (the same as above), Portal', and LoginDB (Table 3) for the fine-grained instance;

where we use strings starting by 'V' to denote variables.

In the coarse-grained specification of the scenario, the login process starts with an invocation by the customer over the `portal.customerLogin` channel. The customer provides his username (`user`), his password (`pwd`), and a private name (`initialId`), which is used to distinguish among invocations by different customers. The complementary request action is performed by the Portal service, and another instance of Portal is spawned to serve possible invocations from further customers. Since the login database service is not present in the current scenario, the task to decide whether a login attempt is successful or not is fully delegated to the portal. The outcome of

```

Customer() = [user] [pwd] [initialId](
  (portal.customerLogin!<user, pwd, initialId>,1.0)
  | (
    (portal.loginKo?<initialId>,1.0).nil
    + [VmyKey](portal.loginOk?<initialId, VmyKey>,1.0).(
      (initialId.customerLoginOk!<loginOk>,1.0)
    )
  )
)

```

Table 1. Definition of Customer for both the coarse- and the fine-grained specifications.

```

Portal() = [VtheUser][VthePwd][VtheId](
  (portal.customerLogin?<VtheUser, VthePwd, VtheId>,1.0).(
    [lkey][ndc](
      (ndc.ndc!<ndc>,1.0)
      | (
        (ndc.ndc?<ndc>,failRate).
        (portal.loginKo!<VtheId>,1.0)
        + (ndc.ndc?<ndc>,okRate).
        (portal.loginOk!<VtheId, lkey>,1.0)
      )
    )
  | Portal()
)

```

Table 2. Definition of Portal for the coarse-grained specification.

the login attempt is abstracted as the mutually exclusive choice between two internal communications over the `ndc.ndc` channel. In case of successful login, a private name (`lkey`) is sent to the customer. This private name models a session identifier, which will be persistently used in further interactions between the customer and any additional service provided by the bank. In the definition of `Portal`, most of the actions are given basic rate `1.0`. Here we observe that this is not the case for the two receiving actions on `ndc.ndc` which are associated with the symbolic values `failRate` and `okRate`. As explained below, these values can be eventually instantiated after the generation of both the labelled transition system and the Markov Chain. This feature, which takes advantage of the PRISM built-in facilities, allows us to model check different configurations of the same sCOWS specification re-using the same PRISM model and just tuning the values of symbolic basic rates.

In the fine-grained specification, the decision whether an attempted login is successful or not is taken by `Portal` on the basis of the outcome of its interactions with the login database `LoginDB`. The portal service forwards to `LoginDB` the credentials received by the `Customer`, and conversely sends the response back to the relevant customer. As in the coarse-grained instance, the request actions involved in the outcome of the login attempt are associated with symbolic basic rates.


```

Portal'() = [VtheUser][VthePwd][VtheId](
  (portal.customerLogin? <VtheUser, VthePwd, VtheId>,1.0).(
    [lkey](
      (db.login!<lkey, VtheUser, VthePwd>,1.0)
      | (
        (db.userKo?<lkey>,1.0).(portal.loginKo!<VtheId>,1.0)
        + (db.userOk?<lkey>,1.0).(
          (portal.loginOk!<VtheId, lkey>,1.0)
          | (portal.creditRequest?<lkey>,1.0)
            .(portal.createInst!<lkey>,1.0)
        )
      )
    )
  )
  | Portal'()
)

LoginDB() = [VrespKey][Vusr][Vpwd](db.login?<VrespKey, Vusr, Vpwd>,1.0).(
  [nd1][ch1](
    (nd1.ch1!<ch>,1.0)
    | (
      (nd1.ch1?<ch>,okRate).
      (db.userOk!<VrespKey>,1.0)
      +(nd1.ch1?<ch>,failRate).
      (db.userKo!<VrespKey>,1.0)
    )
  )
  | LoginDB()
)

```

Table 3. Definition of Portal' and of LoginDB for the fine-grained specification.

3.2 Probabilistic model checking in sCOWS

Probabilistic model checking of sCOWS specifications is based on a Java tool called sCOWS_LTS that provides input to PRISM, a probabilistic model checker for the analysis of properties expressed in CSL (Continuous Stochastic Logic).

The developed tool first implements the sCOWS operational semantics to get the Labelled Transition System (LTS) associated with the input service. For instance, for three parallel customers, Figure 4 and Figure 5 show the LTSs of the fine- and of the coarse-grained specifications, respectively. For the sake of readability, the labels associated with transitions are omitted by both the two graphs. Apart from the lower number of nodes in the coarse-grained case, notice that both LTSs have the same number of absorbing states, representing the possible final configurations of the global service that only depend on the number of successfully logged on customers (0, ..., 3).

The second computational phase of the developed tool consists in the generation of the Continuous Time Markov Chain (CTMC) in PRISM notation. To ease expressing and checking relevant quantitative properties, PRISM allows the association of CTMC states with variables. In order to take full advantage of this feature, when the LTS is translated into the CTMC, the user can specify which sort of actions are relevant to the

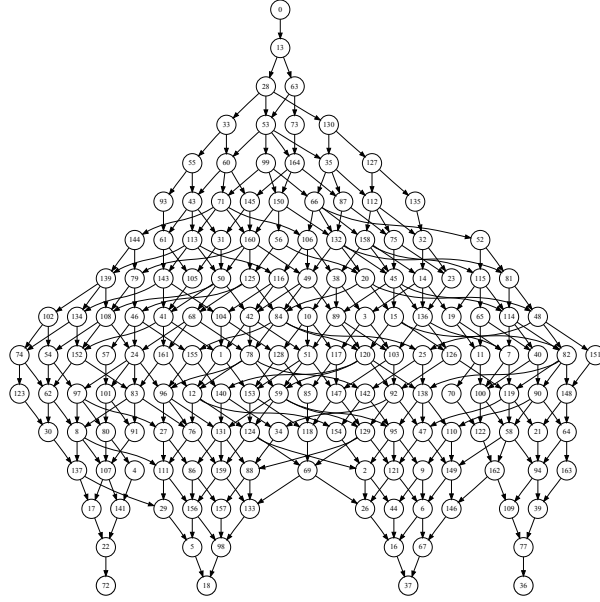


Fig. 4. LTS of the fine-grained specification (labels omitted).

analysis he intends to carry on. For example, the user can choose to associate the communication over a given channel with the increment of an integer PRISM variable. This is indeed what was done for the examples of analysis that are shown below. In those cases we were interested in checking properties depending on the number of successfully logged on customers. Then we used the variable `finished` to trace the number of communications over the channel `portal.loginOk`.

Examples of properties against which sCOWS models can be checked are given below. The first one (1) is a property for *transient* analysis, and the second one (2) is a *steady-state* property, namely they refer, respectively, to the transient evolution of the system, and to its behaviour at steady-state. Formally:

$$P_{=?}[\text{true } \mathcal{U}[T, T] \text{ finished} = N] \quad (1)$$

i.e., “What is the probability that exactly N customers are logged on at time T ?”, and

$$S_{=?}[\text{finished} = N] \quad (2)$$

i.e., “What is the long-run probability of being in a state where exactly N customers are logged on?”. Relatively to (2), notice that, since in the configuration at hand there is no steady-state, the use of the CSL S operator expresses the long-run probability of reaching one of the final configurations of the system. Also, the results obtained by checking (2) represent the asymptotic view of the results for the transient analysis of (1).

Exploiting the support for symbolic rates, we checked our specifications with two different instances of the pair (`okRate`, `failRate`). In particular, we considered the

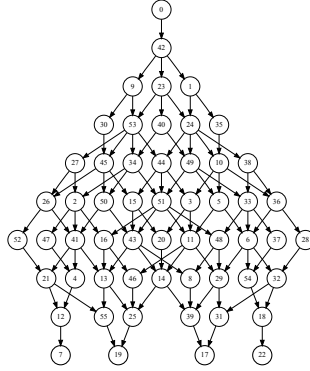


Fig. 5. LTS of the coarse-grained specification (labels omitted).

Logins	0	1	2	3
Probability	$\left(\frac{r_f}{r_{ok}+r_f}\right)^3$	$3\left(\frac{r_{ok}}{r_{ok}+r_f}\right)\left(\frac{r_f}{r_{ok}+r_f}\right)^2$	$3\left(\frac{r_{ok}}{r_{ok}+r_f}\right)^2\left(\frac{r_f}{r_{ok}+r_f}\right)$	$\left(\frac{r_{ok}}{r_{ok}+r_f}\right)^3$

Table 4. Parametric probabilities to get a particular final configuration (r_{ok} stays for `okRate`, and r_f stays for `failRate`).

pair (`okRate` = 4.0, `failRate` = 2.0) where the two values are much similar, and the pair (`okRate` = 30.0, `failRate` = 2.0) in which the `okRate` is fifteen times bigger than `failRate`. The results of the analysis are collectively reported in Figure 6. As expected, for both the two instances of the pair (`okRate`, `failRate`), the fine-grained specification shows to be slower than the coarse-grained one. Notice however that the additional transitions of the fine-grained model are executed before deciding upon the success or the failure of the login attempt, and hence belong to the transient behaviour of the global system, while property (2) refers to steady-state. For this reason, once fixed specific values for the pair (`okRate`, `failRate`), one would expect comparable results for both the coarse- and the fine-grained specifications. In fact, the probability to reach a particular final configuration is driven by a binomial distribution $B\left(3, \frac{\text{okRate}}{\text{okRate}+\text{failRate}}\right)$ which only depends on the values assigned to the symbolic rates (`okRate`, `failRate`).

Logins	(okRate = 4.0, failRate = 2.0)			(okRate = 30.0, failRate = 2.0)		
	Fine	Coarse	Analytical	Fine	Coarse	Analytical
0	0.0370369	0.0370370	0.037	2.441×10^{-4}	2.441×10^{-4}	$2.44140625 \times 10^{-4}$
1	0.2222216	0.2222220	0.2	1.09863×10^{-2}	1.09863×10^{-2}	$1.0986328125 \times 10^{-2}$
2	0.4444432	0.4444438	0.4	0.1647944	0.1647947	0.164794921875
3	0.2962957	0.2962958	0.296	0.8239731	0.8239734	0.823974609375

Table 5. Long-run probabilities of being in one of the absorbing states in the different instances of the two specifications.

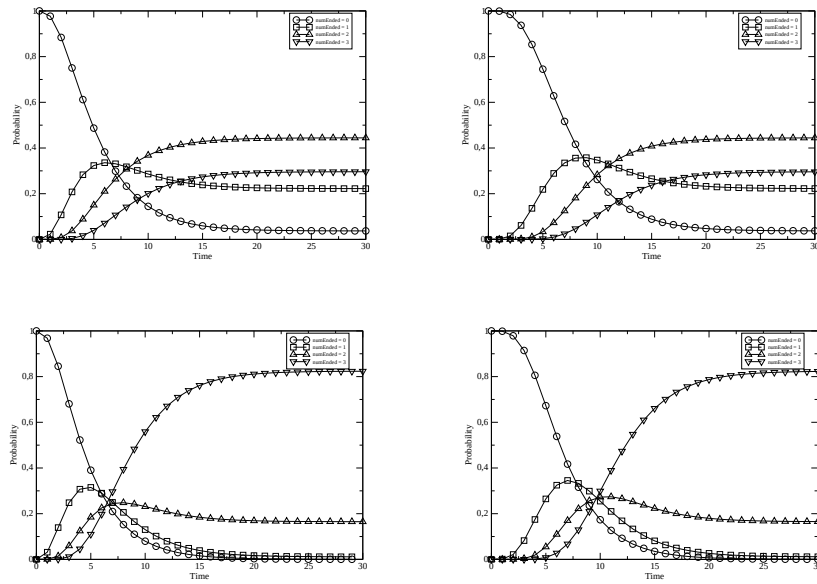


Fig. 6. Long-run probability that N customers are successfully logged on at time T : basic rates ($\text{okRate} = 4.0$, $\text{failRate} = 2.0$) (top) and ($\text{okRate} = 30.0$, $\text{failRate} = 2.0$) (bottom); coarse-grained specification (left) and fine-grained specification (right).

Table 5 sums up the experimental data we obtained when checking (2) and compares them to the theoretical analytical values resulting from the instantiation of the expressions in Table 4. The numerical values obtained by the model checker are approximated to seven digits. There are a few discrepancies (always bound by 10^{-6}) between corresponding experimental and theoretical data. We ascribe this shift to the rounding errors introduced by the model checker computations.

3.3 Approximate model checking in sCOWS

Despite of the many optimizations implemented in `sCOWS_LTS`, on some occasions the state space of a specification can be so big that the generation of the CTMC, as well as the application of numerical algorithms to solve the chain, become really demanding tasks. To face this problem and check properties of sCOWS services without generating their complete transition systems, we also developed a tool for approximate statistical model checking. The tool, called `sCOWS_AMC`, is based on the generation of simulation traces obtained by running a Monte Carlo algorithm. At high level, given a formula ϕ , its model checking consists in the following steps:

- computation of a number of execution traces obtained by direct simulation of the source sCOWS specification;
- evaluation of ϕ against each of the traces;
- statistical reasoning based both on the number of samples performed and on the desired error threshold.

Our tool currently supports the verification of CSL transient time-bounded path formulas like, for instance, the following ones:

$$P_{\geq 0.8}[\text{finished} < 2 \ \mathcal{U}[10, 20] \ \text{finished} \geq 2] \quad (3)$$

i.e., “Is there a probability of at least 80% that in the execution time range $[10, 20]$ the number of logged-on customers varies from a value less than two to a value greater or equal two?”,

$$P_{\gamma}[\text{finished} < 2 \ \mathcal{U}[10, 20] \ \text{finished} \geq 2] \quad (4)$$

i.e., “What is the probability that in the execution time range $[10, 20]$ the number of logged-on customers varies from a value less than two to a value greater or equal two?”

The truth value of CSL probabilistic formulas of the type of (3) is calculated by means of the *sequential probability ratio test* [15] and is inspired by the same approach used Ymer [16]. This method requires to perform a sequence of observations of the hypothesis to be tested. After each observation an error estimation is made, taking into account the results of all the previous observations. When a given error threshold is crossed, the hypothesis is either accepted or rejected. In our case, performing an observation corresponds to testing the formula over an execution trace, which is generated on demand. This approach does not involve the estimation of the probability with which the property is verified: it only checks whether the probability lies below or beyond the given threshold.

Statistical checking of probability estimation formulas, like e.g. (4), requires a different technique which is based on the choice of two parameters: the *approximation*

Coarse-grained specification				
Customers	State space size	LTS generation time	Model Checking time	Simulation time
2	20	1.4	1.5	417.3
3	55	3.5	1.5	988.7
4	125	25.3	1.4	1,979.5
5	251	287.7	3.4	2,801.0
Fine-grained specification				
Customers	State space size	LTS generation time	Model Checking time	Simulation time
2	44	3.2	1.5	1,125.6
3	164	22.5	1.5	3,271.8
4	494	310.6	7.4	4,958.9
5	1,286	48,420.5	69.5	11,734.3

Table 6. Comparative view of the model checking approach and the statistical checking approach (time expressed in seconds).

parameter ε , and the *confidence parameter* δ . As in the above case, the analysis is computed on a series of observations on random walks over the LTS. This time, however, the number of observations necessary to obtain the desired approximation level is estimated *before* observations are made. As shown in [5], taking a sample of

$$4 \times \frac{\log \frac{2}{\delta}}{\varepsilon^2}$$

observations, ensures that the probability that the obtained estimation diverges from the real value by more than ε is less than δ . Also, the complexity of this probability estimation method is linearly dependent on the number of observations, and hence on $\log \frac{1}{\delta}$ and on $\frac{1}{\varepsilon^2}$. So the value of the approximation parameter has to be carefully selected. Given the best-matching communication policy of sCOWS, all the possible transitions exiting from a state must be computed on the fly at each step of the simulation. Hence random walks over sCOWS specifications are more costly than over CTMCs. To get a good balance between approximation value and execution time, we carried out our experiments by choosing $\delta = 10^{-1}$ and $\varepsilon = 10^{-2}$, so testing properties over 52042 execution traces.

3.4 Comparison between the CTMC- and the simulation-based approaches

Table 6 shows a comparative view of the results we obtained by applying probabilistic and statistical model checking. Comparison is in terms of execution time. As expected, for the coarse-grained specification the simulation-based method is less efficient than the other. This is due to the fact that the state space is not very large, and thus relatively cheap to build. Conversely, when the number and the complexity of the involved services increase, the method based on the generation of the complete LTS gets outperformed by the approach used in sCOWS-AMC.

4 Analysis in PEPA

We turn now to modelling of the credit request scenario in PEPA [6] and analysis of the model using the PEPA software tools developed within the SENSORIA Project [3, 14].

We begin with a description of the bank’s customers and the process which they follow in order to secure a loan from the bank. This presents the customer’s view of the process in terms of activities and choices along the way.

The customer’s first action is to initiate a loan (*request*). To carry this through they must enter their balance data and securities (*enterData*) and send this to the credit portal with an XBRL upload (*uploadData*). (XBRL is the eXtensible Business Reporting Language.) The customer then waits to see if the request will be approved or declined (the *approve* and *decline* activities respectively). If the request is approved the customer has no further business and the next waiting customer can be considered. If the request is declined the customer can try again (*reapply*) and will do so with probability t_0 . If they do not wish to reapply they can yield to the next customer.

This behaviour is described in PEPA thus. A rate is associated with each activity and a probability is associated to this when the customer is making the decision of what course of action to take next.

$$\begin{aligned}
 \text{Customer} &\stackrel{\text{def}}{=} (\text{request}, r_{\text{request}}).\text{Entering} \\
 \text{Entering} &\stackrel{\text{def}}{=} (\text{enterData}, r_{\text{enterData}}).\text{Upload} \\
 \text{Upload} &\stackrel{\text{def}}{=} (\text{uploadData}, r_{\text{upload}}).\text{Wait} \\
 \text{Wait} &\stackrel{\text{def}}{=} (\text{approve}, \top).\text{Customer} \\
 &\quad + (\text{decline}, \top).\text{Decide} \\
 \text{Decide} &\stackrel{\text{def}}{=} (\text{reapply}, r_{\text{reapply}} \times t_0).\text{Entering} \\
 &\quad + (\text{reapply}, r_{\text{reapply}} \times t_1).\text{Customer}
 \end{aligned}$$

The service is a reactive system. It takes no action until the upload of XBRL data is complete. At this point it validates the data by using a validation web service which determines whether or not the balance data is valid (*validateData*). We are not interested in the processing of invalid data in this scenario and so the next behaviour which we model is passing the valid data to the bank (*sendBank*). The service is then ready to receive the next request.

$$\begin{aligned}
 \text{Service} &\stackrel{\text{def}}{=} (\text{uploadData}, \top).\text{Validate} \\
 \text{Validate} &\stackrel{\text{def}}{=} (\text{validateData}, r_{\text{validate}}).\text{SendBank} \\
 \text{SendBank} &\stackrel{\text{def}}{=} (\text{sendBank}, r_{\text{sendBank}}).\text{Service}
 \end{aligned}$$

The relevant business functions of the bank are expressed in the *Bank* component. This documents the “predecision” phase which can have three possible outcomes. Some applications can be immediately approved, and others immediately declined. Some proportion need to be processed by a bank employee who will either decline the loan, or

approve it (which requires confirmation, which may or may not be forthcoming).

$$\begin{aligned}
Bank &\stackrel{def}{=} (sendBank, \top).PreDecide \\
PreDecide &\stackrel{def}{=} (predecide, r_{predecide} \times p_0).Approve \\
&\quad + (predecide, r_{predecide} \times p_1).Decline \\
&\quad + (predecide, r_{predecide} \times p_2).Employee \\
Employee &\stackrel{def}{=} (decide, r_{decide} \times q_0).Confirm \\
&\quad + (decide, r_{decide} \times q_1).Decline \\
Confirm &\stackrel{def}{=} (decide, r_{decide} \times s_0).Approve \\
&\quad + (decide, r_{decide} \times s_1).Decline \\
Approve &\stackrel{def}{=} (approve, r_{inform}).Bank \\
Decline &\stackrel{def}{=} (decline, r_{inform}).Bank
\end{aligned}$$

To complete the model description we need to compose the sequential components defined above and require them to cooperate on their shared activities. This leads to the following model composition, which defines the initial state of the model.

$$Customer \bowtie_{\{uploadData, approve, decline\}} (Service \bowtie_{\{sendBank\}} Bank)$$

In order to numerically evaluate this model it is necessary to assign particular values to the rates and probabilities used in the model. These values can be found in Tables 7 and 8. A screenshot of the PEPA Eclipse Plug-in processing the model can be found in Figure 7.

Customer rates	Service rates	Bank rates
$r_{request} = 3.11944$	$r_{validate} = 1.43141$	$r_{predecide} = 5.15757$
$r_{enterData} = 0.04667$	$r_{sendBank} = 1.53785$	$r_{decide} = 0.01221$
$r_{upload} = 0.88424$		$r_{inform} = 0.45729$
$r_{reapply} = 0.02036$		

Table 7. Table of rate values used in the model. All rates are expressed at the granularity of minutes. The reciprocal of the rate gives the mean or expected value of the duration of the activity. Thus, the average time for one bank employee to decide on a loan is about 82 minutes ($1/r_{decide} = 81.90008$).

Predecision	Employee decision	Supervisor decision	Reapplication
$p_0 = 0.20907$	$q_0 = 0.17441$	$s_0 = 0.56423$	$t_0 = 0.08970$
$p_1 = 0.32075$	$q_1 = 0.82559$	$s_1 = 0.43577$	$t_1 = 0.91030$
$p_2 = 0.47018$			

Table 8. Table of probability values used in the model. Each column sums to 1.

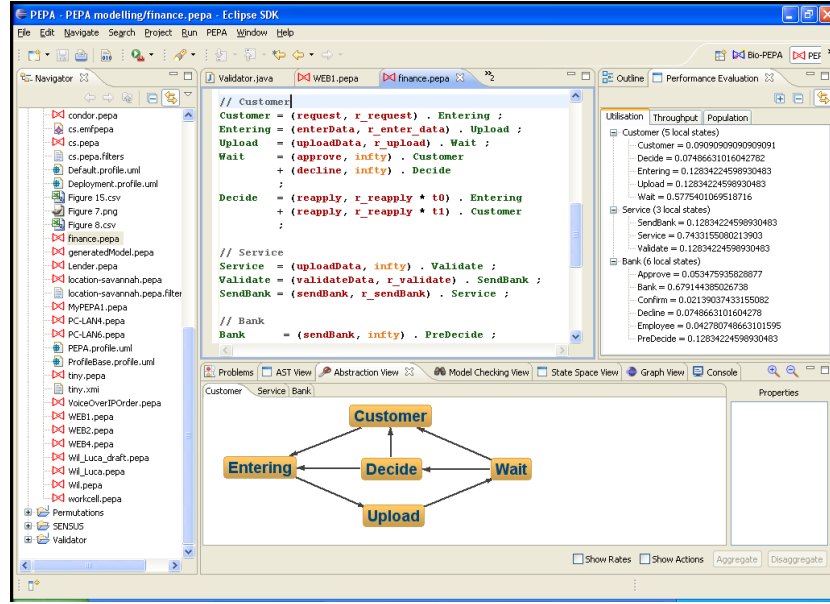


Fig. 7. Screenshot of the PEPA Eclipse Plug-in processing the credit request scenario. The PEPA model is displayed in the editor and the Abstraction view gives a graphical presentation of the *Customer* component. Performance results are shown in the Performance Evaluation view.

The computational difficulty of the problem is compounded as usual in service-oriented computing because we face the problem of being uncertain about which instance of a service centre will be selected at run-time. Because of this we must evaluate the model not just one time but repeatedly, once for each possible binding of rate parameters to rate constants. This *parameter sweep* must consider the full combinatorial potential of rate parameter assignments and the credit portal must satisfy its service-level agreement for every possible parameter assignment.

We queried the model using the XSP performance query language [4]. The query which we applied was the following:

$$request:start, (approve \mid decline):stop$$

Informally, we can think of this query as starting a clock when a request arrives and stopping this clock when the request is either approved or declined. More formally, this query defines a set of start states (all those states reachable via activity *request*), a set of end states (all those states reachable via activities *approve* or *decline*), and the set of all paths through the model's underlying labelled transition system from start states to end states. The analysis results computed by the PEPA tools deliver the probability of completing the passage from the start states to the end states for any given time bound. We present these results in Figure 8.

One of the uses of these results is to allow us to identify those activities which are bottlenecks for the passage of interest. By varying the rates in the model we can

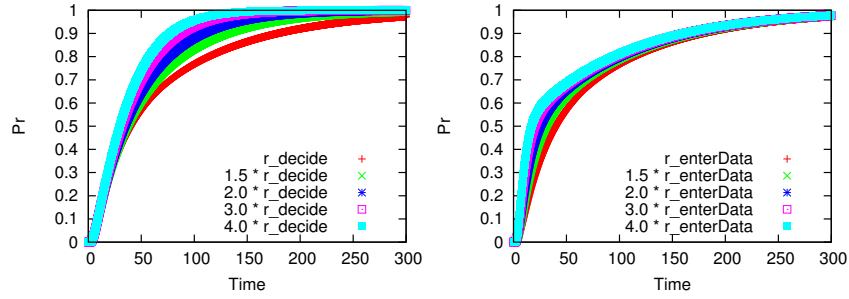


Fig. 8. Two cumulative distribution functions showing the effect of scaling rates r_decide and $r_enterData$

experiment to find that changes to the rate of entering data ($r_enterData$) have little impact—see the right-hand graph in Figure 8. In contrast, changes in the rate of human decision making (r_decide) have a much greater impact—compare the left-hand graph in Figure 8. From investigations such as these we can decide where effort would be best spent in improving functions within the entire business process.

5 Concluding remarks

We presented a number examples of quantitative analysis of scenarios taken from the SENSORIA Finance Case-Study. The shown applications are based on tools developed within the Project for three distinct stochastic process calculi: MarCaSPiS, sCOWS, and PEPA.

The tool SoSL-MC, built around the MRMC model checker, allows checking MarCaSPiS specifications against properties expressed in SoSL, a service oriented stochastic logics. The examples presented in the paper show how the distribution of clients among the three possible rating classes influences system performance.

Relative to sCOWS, two tools have been illustrated. They adopt distinct approaches to the model checking of CSL formulas. The tool sCOWS_LTS generates the transition system corresponding to the given sCOWS specification, then converts it into a CTMC to be used as input for PRISM. In sCOWS_AMC, instead, statistical reasoning is applied to get the wanted results (possibly with a small approximation error) starting from simulations rather than from LTSs. The two tools have been applied to compare the long-run probability of successful login for both different rates in the specifications and different implementation of the login procedure. Also, the range of applicability of the two tools has been briefly commented upon in a comparative way.

The latest kind of analysis reported in the paper was the one carried out using PEPA and the query language XSP of stochastic probes. This setting has been used to perform sensitivity analysis over the basic rates associated to actions embedding the waiting time for loan approval or decline.

References

1. Adnan Aziz, Kumud Sanwal, Vigyan Singhal, and Robert Brayton. Model-checking continuous-time markov chains. *ACM Trans. on Computational Logic*, 1(1):162–170, 2000.
2. Michele Boreale, Roberto Bruni, Rocco De Nicola, and Michele Loreti. Sessions and pipelines for structured service programming. In Gilles Barthe and Frank S. de Boer, editors, *Proc. of FMOODS'08*, volume 5051 of *Lecture Notes in Computer Science*, pages 19–38. Springer, 2008.
3. Allan Clark. The ipclib PEPA Library. In Mor Harchol-Balter, Marta Kwiatkowska, and Miklos Telek, editors, *Proceedings of the 4th International Conference on the Quantitative Evaluation of SysTems (QEST)*, pages 55–56. IEEE, September 2007.
4. Allan Clark and Stephen Gilmore. State-aware performance analysis with eXtended Stochastic Probes. In Nigel Thomas and Carlos Juiz, editors, *Proceedings of the 5th European Performance Engineering Workshop (EPEW 2008)*, volume 5261 of *LNCS*, pages 125–140, Palma de Mallorca, Spain, September 2008. Springer.
5. Thomas Héroult, Richard Lassaigne, Frédéric Magniette, and Sylvain Peyronnet. Approximate probabilistic model checking. In *Proc. 5th Int. Conf. on Verification, Model Checking and Abstract Interpretation (VMCAI'04)*, volume 2937 of *LNCS*, pages 307–329. Springer, 2004.
6. Jane Hillston. *A Compositional Approach to Performance Modelling*. Cambridge University Press, 1996.
7. Joost-Pieter Katoen, Maneesh Khattri, and Ivan S. Zapreev. A Markov reward model checker. In *Quantitative Evaluation of Systems (QEST)*, pages 243–244. IEEE CS Press, 2005.
8. Marta Kwiatkowska, Gethin Norman, and David Parker. PRISM: Probabilistic Model Checking for Performance and Reliability Analysis. *ACM SIGMETRICS Performance Evaluation Review*, 36(4):40–45, 2009.
9. Alessandro Lapadula, Rosario Pugliese, and Francesco Tiezzi. Calculus for Orchestration of Web Services. In *Proc. ESOP'07*, volume 4421 of *LNCS*, pages 33–47, 2007.
10. Robin Milner. *Communicating and mobile systems: the π -calculus*. Cambridge University Press, 1999.
11. Davide Prandi and Paola Quaglia. Stochastic COWS. In B.J. Krämer, K.-J. Lin, and P. Narasimhan, editors, *Proc. 5th International Conference on Service Oriented Computing, ICSOC '07*, volume 4749 of *LNCS*, pages 245–256. Springer, 2007.
12. Davide Sangiorgi and David Walker. *The π -calculus: a Theory of Mobile Processes*. Cambridge University Press, 2001.
13. Stefano Schivo. *Statistical model checking of Web Services*. PhD thesis, Int. Doctorate School in Information and Communication Technologies, University of Trento, 2010.
14. Mirco Tribastone, Adam Duguid, and Stephen Gilmore. The PEPA Eclipse Plug-in. *Performance Evaluation Review*, 36(4):28–33, March 2009.
15. Abraham Wald. Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16(2):117–186, 1945.
16. Håkan L. Younes. Ymer: A statistical model checker. In *Computer Aided Verification*, pages 429–433, 2005.